Ollscoil Teicneolaíochta an Oirdheiscirt
South East Technological University

AI Principles and Practice · COMP-0484

Data Analysis and the ML Workflow

Week 1 · Reference Booklet

A plain-English guide: theory, mathematics, runnable Python, and outputs

Lecturer: Indrakshi

Department of Computing and Mathematics

How to use this booklet

Welcome. This booklet is the long, friendly companion to the Week 1 lecture. The slides move fast; this document slows everything down and explains each idea as if we were sitting together at a whiteboard. You do not need to have seen any of this before. Where a word is technical, we say what it means in ordinary language the first time it appears.

Every topic is built the same way, so you always know what to expect:

1. **A plain explanation** of what the idea is and why anyone should care about it.
2. **The mathematics**, written out as a formula, immediately followed by a sentence that reads the formula aloud in words.
3. **A short Python program** you can actually run, with a note on what each part does.
4. **The command to run it**, shown on a dark line beginning with a `$` prompt.
5. **The output**, shown in a dark box, with a short note on how to read it.
6. **A picture**, whenever a picture explains more than a paragraph could.

Everything in this booklet is reproducible. A single script, `make_assets.py`, recreates every number and every figure you see here. It uses a fixed "random seed", which means the random numbers come out the same way every time, so your results will match the booklet exactly.

Setting up the tools (once)

You need Python and a handful of free libraries. In a terminal, run the three lines below. The first line makes a clean, private workspace for this module (a "virtual environment") so these installs do not interfere with anything else on your machine.

terminal python

```
python -m venv .venv && source .venv/bin/activate
pip install numpy pandas scikit-learn scipy matplotlib
python make_assets.py        # recreates every figure and printed output
```

What the libraries are for

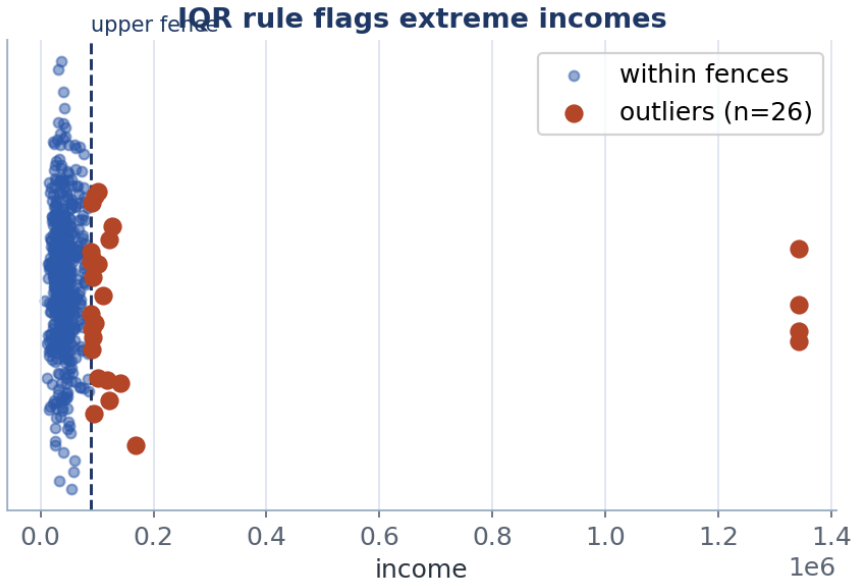
NumPy does fast maths on arrays of numbers. **pandas** is the spreadsheet-in-Python that holds our table of data. **scikit-learn** is the standard machine-learning toolkit. **SciPy** adds statistics. **Matplotlib** draws the charts.

Contents

1 The end-to-end machine-learning workflow

Before we touch a single number, it helps to see the whole journey. Every machine-learning project, from a student exercise to a system used by millions, follows the same handful of steps in the same order. Knowing the steps tells you where you are, what comes next, and where a mistake is likely to creep in.

> Think of building a machine-learning model like cooking a meal for guests. First you decide what to cook (the goal). Then you buy ingredients (the data) and inspect them to check they are fresh (exploration). You wash and chop (cleaning). You cook (training the model). You taste before serving (evaluation). You serve it (deployment). And you watch your guests' faces to learn for next time (monitoring). Skip the inspecting and chopping, and no amount of clever cooking will rescue the meal.



The eight stages of the workflow. The dashed arrow underneath is the important part: it is a loop, not a straight line, because what you learn at the end changes how you start again.

1.1 The eight stages, one at a time

1. **Frame the problem.** Decide, in plain words, what question the model should answer and what a good answer looks like. "Predict which customers will cancel next month" is a frame. Vague goals lead to useless models.
2. **Collect data.** Gather the information you will learn from, and write down where it came from and whether you were allowed to use it. This record matters later for fairness and the law.
3. **Explore (EDA).** "EDA" stands for **exploratory data analysis**, which is a fancy name for "look carefully at your data before doing anything else". This whole booklet is mostly about this step.
4. **Preprocess.** Clean and reshape the data so a model can use it: fill in blanks, fix odd values, turn words into numbers, and put everything on a comparable scale.
5. **Model.** Choose a method and let it learn the patterns. This is Week 2 onwards.
6. **Evaluate.** Measure honestly how well the model does, using the right yardstick and a fair test. This is Week 2.
7. **Deploy.** Put the model somewhere it actually makes decisions.
8. **Monitor.** Keep watching. Real-world data drifts over time, so a model that was good last year can quietly go stale.

Weeks 1 to 5 of this module live almost entirely in stages 1 to 4. The unglamorous early steps are where most projects are won or lost. Get them right and the modelling becomes easy; get them wrong and no clever model can save you.

1.2 Where this module fits

You will return to this same map all term. Here is which week handles which stage.

Workflow stage	Module coverage
Data, exploration, preprocessing	Week 1 (this booklet)
Modelling with labelled data	Week 2
Ethics, fairness, safety, governance	Weeks 3 and 4
Finding structure without labels	Week 5
Neural networks and the mathematics	Weeks 7 and 8
Modern methods	Weeks 9 to 11

1.3 The running example we will use throughout

To keep things concrete, every example in this booklet uses one small, made-up table of **customers**. We build it with code rather than downloading it, for two reasons: you can see exactly what is in it, and because of the fixed random seed it comes out identically every time.

We deliberately put problems into this data on purpose: a lopsided income column, some blank cells, and a few absurdly large values. These are the exact defects you will meet in real data, so it is worth learning to spot and fix them in a safe setting first.

data.py python

```python
import numpy as np, pandas as pd
from scipy import stats

rng = np.random.default_rng(42)        # fixed seed: same "random" data every run
n = 600                                # 600 pretend customers
df = pd.DataFrame(dict(
    age=rng.normal(40, 12, n).clip(18, 85).round(0),
    income=rng.lognormal(10.6, 0.45, n).round(0),        # deliberately lopsided
    tenure=rng.integers(0, 72, n),                        # months as a customer
    plan=rng.choice(["Basic", "Standard", "Premium"], n, p=[.5, .3, .2]),
    region=rng.choice(["North", "South", "East", "West"], n),
    satisfaction=rng.choice(["Low", "Medium", "High"], n, p=[.2, .5, .3]),
))
df["monthly_charge"] = (20 + 0.00035*df.income + 0.05*df.tenure
                        + rng.normal(0, 4, n)).round(2)
df["churn"] = ((df.satisfaction.eq("Low")*0.5) + (df.tenure.lt(6)*0.3)
               + rng.random(n)*0.45 > 0.6).astype(int)   # 1 = customer left
df.loc[rng.random(n) < 0.08, "income"] = np.nan          # poke holes (blanks)
df.loc[rng.random(n) < 0.05, "monthly_charge"] = np.nan
df.loc[rng.choice(n, 4, replace=False), "income"] = df.income.max() * 8  # 4 absurd values
```

Each row is one customer. Each column is one thing we know about them: their `age`, yearly `income`, how many months they have been a customer (`tenure`), which `plan` they are on, their `region`, their `satisfaction`, their `monthly_charge`, and finally `churn`, which is `1` if they left and `0` if they stayed. That last column, `churn`, is the thing we would eventually like to predict.

Now let us actually look at it. The next snippet prints the first few rows and a quick numeric summary.

profile.py python

```python
print(df.head())                                          # first 5 rows
print(df[["age", "income", "monthly_charge"]].describe().round(2))
```

```
$ python profile.py
```

output

```
    age   income  tenure      plan region satisfaction  monthly_charge  churn
0  44.0  50612.0      29  Standard   West       Medium           47.29      0
1  28.0  30949.0      14  Standard  South       Medium           31.40      0
2  49.0  71218.0       7   Premium  North       Medium           49.39      0
3  51.0      NaN      30     Basic   West         High           31.88      0
4  18.0      NaN      37   Premium  North         High           32.27      0

          age        income   monthly_charge
count  600.00        547.00           570.00
mean    39.82      53565.44            37.39
std     11.42     112607.49             8.36
50%     40.00      40388.00            36.20
max     75.00    1342344.00            84.85
```

How to read this output

The top block is the first five customers. Notice the word `NaN` in the income column for rows 3 and 4. `NaN` means "not a number", which is how pandas writes a blank cell. So straight away we can see some incomes are missing.

The bottom block is a numeric summary. Each row is a fact about a column. `count` is how many values are present: income shows only 547 out of 600, so 53 incomes are missing. `mean` is the average. `std` is a measure of spread (more on this soon). `50%` is the middle value (the median). `max` is the largest value. Three things already look wrong with income: it has missing values, its average (53,565) is far above its middle value (40,388), and its largest value is a ridiculous 1,342,344. We will investigate all three. This five-second glance is exactly the kind of habit that separates careful analysts from careless ones.

2 Data types and encoding

Computers ultimately only do arithmetic, so a model can only learn from numbers. Some of our columns are already numbers (age, income). Others are words (plan, region). Before modelling we must turn the words into numbers, and the way we do that depends on what kind of variable it is. Choosing the wrong conversion can quietly teach the model something false, so this short chapter matters more than it looks.

2.1 The five kinds of variable

Type	Plain meaning	In our data
Binary	Only two possible values	churn (left or stayed)
Categorical (nominal)	Named groups with no order	region, plan
Ordinal	Named groups that do have an order	satisfaction
Discrete	Whole-number counts	tenure (months)
Continuous	Any value on a scale	age, income

The key distinction for the next two sections is **order**. Some categories have a natural ranking ("Low", "Medium", "High" clearly go in that order). Others do not ("North" is not more or less than "South"). We encode these two situations differently.

2.2 Ordinal encoding: when order is real

When categories genuinely have an order, we can replace them with numbers that keep that order. Satisfaction is a perfect example: "Low" then "Medium" then "High".

$$\text{Low} \to 0, \quad \text{Medium} \to 1, \quad \text{High} \to 2$$

the numbers rise as the satisfaction rises, so the order is preserved

Read this as: replace the word "Low" with 0, "Medium" with 1, and "High" with 2. Because 0 is less than 1 is less than 2, the model still "knows" that High beats Medium beats Low. The code below does exactly this. The important detail is that we **tell** the encoder the correct order, rather than letting it guess.

encode_ordinal.py — python

```python
from sklearn.preprocessing import OrdinalEncoder
oe = OrdinalEncoder(categories=[["Low", "Medium", "High"]])   # we state the order
print(oe.fit_transform(df[["satisfaction"]])[:4].astype(int).ravel())
```

Output

```
[1 1 1 2]      # the first four customers were Medium, Medium, Medium, High
```

The output is just the first four satisfaction values translated into numbers. Medium became 1 and High became 2, exactly as our map says.

Watch out
If you do not specify the order, the encoder sorts the words alphabetically, which would give "High" = 0, "Low" = 1, "Medium" = 2. That is nonsense: it tells the model that "Low" sits between "High" and "Medium". Always pass the order yourself.

2.3 One-hot encoding: when there is no order

For categories with no natural order, like region, turning them into 0, 1, 2, 3 would be a lie. The model would believe "West" (say, 3) is three times "North" (0), which is meaningless. The fix is **one-hot encoding**: make one new yes-or-no column per category, and put a 1 in the column that applies.

$$\text{plan} \to [\text{Basic?}, \text{Premium?}, \text{Standard?}\,] \in \{0,1\}^3$$

one column per plan; the customer's plan gets a 1, the rest get 0

Read this as: instead of one "plan" column, create three columns, one for each plan. For a customer on the Standard plan, the Standard column is 1 and the other two are 0. The name "one-hot" comes from exactly one column being "hot" (set to 1) at a time.

encode_onehot.py — python

```python
from sklearn.preprocessing import OneHotEncoder
oh = OneHotEncoder(sparse_output=False, dtype=int, handle_unknown="ignore")
print(oh.get_feature_names_out(["plan"]))
print(oh.fit_transform(df[["plan"]])[:4])
```

Output

```
['plan_Basic' 'plan_Premium' 'plan_Standard']
[[0 0 1]
 [0 0 1]
 [0 1 0]
 [1 0 0]]
```

The first line names the three new columns. Each row after that is one customer: the first two are on Standard (1 in the third column), the third is on Premium, the fourth is on Basic. No false ordering has been introduced.

Two practical notes. If a category has thousands of distinct values (think "postcode"), one-hot encoding creates thousands of columns, which is wasteful; other techniques exist for that case. And `handle_unknown="ignore"` means that if a brand-new category shows up later that the model never saw during training, the program will not crash; it simply marks all the plan columns as 0.

3 Summary statistics

A **summary statistic** is a single number that captures something about a whole column. Instead of staring at 600 incomes, we ask: what is a typical value (the centre)? how spread out are they (the spread)? and is the shape lopsided (the shape)? These three questions, centre, spread and shape, are the backbone of understanding any column. They are wonderfully useful and, as we will see in Chapter 4, occasionally treacherous when used alone.

3.1 Centre: the typical value

There are two everyday ways to say "typical": the **mean** (the ordinary average) and the **median** (the middle value when you line everything up).

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad \text{median} = \text{the middle value once the data is sorted}$$

the mean adds everything up and divides by the count; the median just picks the middle

Read the mean formula as: add up all the values ($\sum$ means "add up"), then divide by n, the number of values. The little bar over the x is read "x-bar" and is the standard symbol for the mean. The median needs no formula: sort the values from smallest to largest and take the one in the middle.

Imagine ten people in a room earning ordinary salaries. The mean and median are close. Now a billionaire walks in. The **mean** salary suddenly leaps to millions, even though nobody in the room except one person is rich. The **median**, the person standing in the middle of the line, barely moves. That is why the median is often the more honest "typical" value when a few extreme values are present.

centre.py — python

```python
inc = df["income"].dropna()            # drop blanks, keep real incomes
print(f"mean   = {inc.mean():,.1f}")
print(f"median = {inc.median():,.1f}")
```

Output

```
mean   = 53,565.4
median = 40,388.0
```

The mean is about 32 percent higher than the median. That gap is the billionaire effect: our few absurd incomes have dragged the average upward, while the median, the middle customer, stayed put. If a manager asked "what does a typical customer earn?", the honest answer here is the median, roughly 40,000, not the average.

3.2 Spread: how varied are the values

Knowing the centre is not enough. Two columns can have the same average yet be wildly different, one tightly clustered and one all over the place. **Spread** measures that. The common measures are the **variance**, its square root the **standard deviation**, and the **interquartile range** (IQR).

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x})^2, \quad s = \sqrt{s^2}, \quad \text{IQR} = Q_3 - Q_1$$

variance is the average squared distance from the mean; IQR is the width of the middle half

Read the variance as: for each value, measure how far it is from the mean $(x_i - \overline{x})$, square that distance so negatives do not cancel positives, add them all up, and average. We divide by $n-1$ rather than n for a subtle statistical reason (it corrects a small bias); you can treat it as "almost the average". The standard deviation s is just the square root of the variance, which conveniently puts it back into the original units (euros, not euros-squared). The IQR is simpler: Q_1 is the value below which the lowest 25 percent sit, Q_3 is the value below which the lowest 75 percent sit, and the IQR is the distance between them, covering the middle 50 percent of customers and ignoring the extremes.

spread.py — python

```python
q1, q3 = inc.quantile([.25, .75])      # the 25% and 75% marks
print(f"std = {inc.std(ddof=1):,.1f}")
print(f"IQR = {q3 - q1:,.1f}  (Q1={q1:,.0f}, Q3={q3:,.0f})")
```

Output

```
std = 112,607.5
IQR =  23,204.5  (Q1=30,143, Q3=53,348)
```

Look at how different these two spread measures are. The standard deviation (112,607) is almost five times the IQR (23,204). The standard deviation, like the mean, is heavily inflated by the few absurd incomes, because squaring a huge distance makes it enormous. The IQR ignores those extremes entirely and reports a calm 23,204, which is a far more honest picture of how much ordinary customers differ. When data has outliers, prefer the IQR.

3.3 Shape: is the data lopsided or top-heavy

The final question is the **shape** of the data. Two numbers capture it. **Skewness** measures lopsidedness: is there a long tail stretching off to one side? **Kurtosis** measures how prone the data is to extreme values, that is, how heavy its tails are compared to a tidy bell curve.

$$g_1 = \frac{\frac{1}{n}\sum (x_i - \overline{x})^3}{s^3}, \qquad g_2 = \frac{\frac{1}{n}\sum (x_i - \overline{x})^4}{s^4} - 3$$

skewness uses cubes (sensitive to one-sided tails); kurtosis uses fourth powers (sensitive to extremes)

You do not need to compute these by hand, but here is the intuition. Cubing the distances in g_1 keeps their sign, so a long tail on the right (big positive distances) makes the skew positive; a long tail on the left makes it negative; a symmetric column gives roughly zero. Raising to the fourth power in g_2 makes very large distances dominate, so heavy tails (lots of extreme values) give a large kurtosis. We subtract 3 so that a perfect bell curve scores 0.

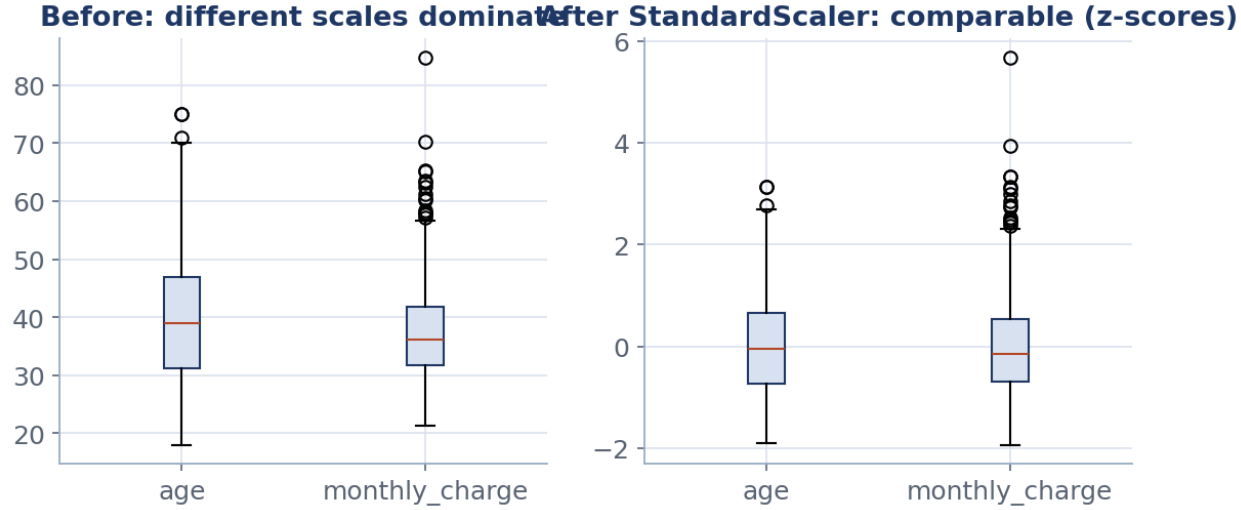
shape.py — python

```python
from scipy import stats
print(f"skew = {stats.skew(inc):.2f}")
print(f"kurt = {stats.kurtosis(inc):.2f}   # 'excess' kurtosis, bell curve = 0")
```

Output

```
skew = 10.99
kurt = 122.93
```

A neat, symmetric column would score near 0 on both. Ours scores 10.99 and 122.93, which is enormous. Translated into English: income is extremely lopsided to the right and riddled with extreme values. The statistics are simply confirming, with numbers, what we suspected from the `describe()` summary. In Chapter 6 we will track down and treat the culprits.

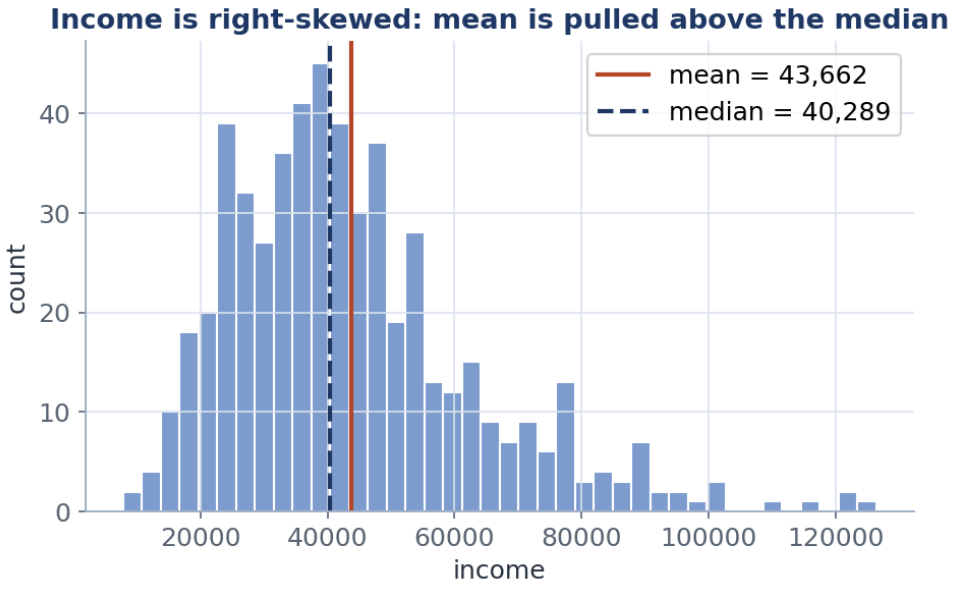


Three shapes for comparison. The middle one is symmetric (skew near 0). The right one has a long tail to the right (positive skew); the left one has a long tail to the left (negative skew). The sign of the skew always points toward the long tail.

3.4 Seeing a column at a glance

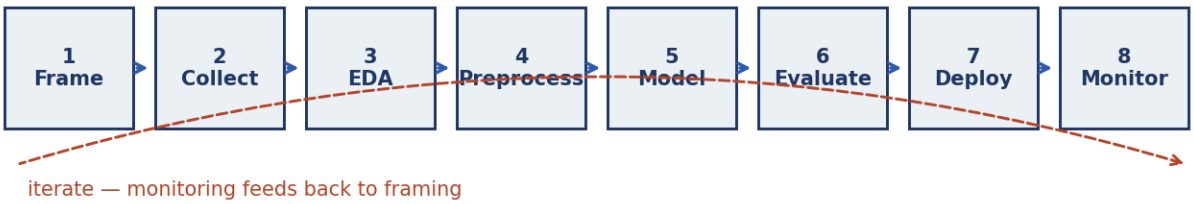
Numbers are precise but a picture is faster. Two charts do most of the work in practice.

A **box plot** draws the five-number summary as a box and whiskers. It is the quickest way to spot the centre, the spread, and any outliers all at once.



Reading a box plot. The box spans the middle 50 percent (Q1 to Q3); the line inside is the median; the whiskers reach out to 1.5 times the IQR; any dots beyond the whiskers are flagged as possible outliers.

A **histogram** slices the range into bins and counts how many values fall in each, so you can see the whole shape, not just a few summary numbers.



Income, viewed up to its 99th percentile so the chart is readable. The red line is the mean and the dashed navy line is the median. You can see the mean has been pulled to the right of the median by the long tail. This is the billionaire effect, drawn out.

4 Distributions and visualisation

A **distribution** simply describes how common each value is in a column. If you imagine sorting every customer's income into buckets and counting how many land in each bucket, the resulting shape is the distribution. Understanding distributions is what lets you say sentences like "most customers earn around 40,000, but a few earn far more".

4.1 The Normal distribution (the bell curve)

The most famous distribution is the **Normal**, also called the **Gaussian** or the **bell curve**. It is the smooth, symmetric hump you have seen on exam-grade charts. It is special because it shows up everywhere in nature and because it is completely described by just two numbers: its mean (where the hump sits) and its standard deviation (how wide the hump is).

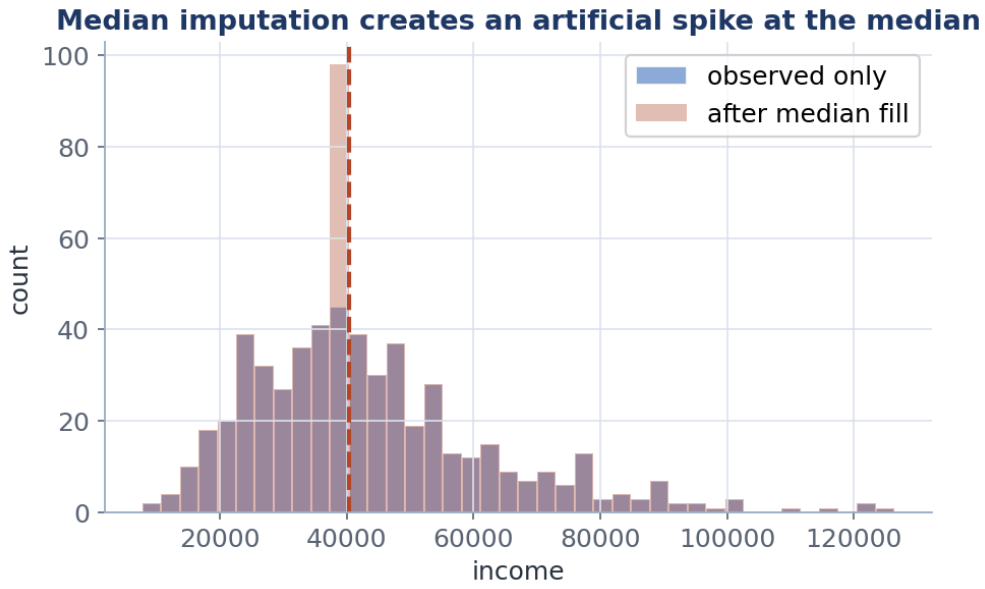
$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

the height of the bell at any point x, controlled entirely by the mean mu and spread sigma

You will never type this formula by hand, so do not be alarmed by it. The single idea to take away is that a Normal distribution is pinned down by only its centre (μ, "mu", the mean) and its spread (σ, "sigma", the standard deviation). The important warning for real work is this: most real-world columns are **not** Normal. Income certainly is not. So never assume the bell curve without first looking at a histogram.

4.2 Why summary numbers are not enough

Here is the single most important lesson of this chapter. Two columns can have the **exact same** mean and the exact same standard deviation and still be completely different shapes. If you only ever look at the summary numbers, you would never know.



Both columns have the same mean and the same standard deviation. The left one is a single hump; the right one has two separate humps. The numbers cannot tell them apart, but your eyes can in a second. This is why you always plot.

The practical rule that follows is simple: always draw the picture. A two-second histogram will catch problems that no table of statistics can.

4.3 Choosing the right chart

Visualisation is a skill, not decoration. The trick is to pick the chart that answers your question. Here is a cheat-sheet you can lean on.

What you want to know	Chart to use
The shape of one column	Histogram, density curve, or box plot
Whether two columns move together	Scatter plot
How groups compare	Grouped bar chart, or box plot per group
How something changes over time	Line chart
Relationships among many columns	Correlation heatmap or pair plot

Whatever you draw, label the axes honestly, start counts at zero where it matters, and be suspicious of fancy colour schemes that make small differences look dramatic.

5 Correlation

Correlation answers a very natural question: when one thing goes up, does another tend to go up too? Taller people tend to weigh more; hotter days tend to sell more ice cream. Correlation puts a single number on that "tend to". It is one of the most useful tools in data analysis, and also one of the most misunderstood, so we will treat it carefully.

5.1 Pearson correlation: do they move in a straight line

The standard measure is the **Pearson correlation**, written r. It ranges from -1 to $+1$. A value near $+1$ means "when one goes up, the other goes up, in a tidy straight line". Near -1 means "when one goes up, the other goes down". Near 0 means "no straight-line relationship".

$$r = \frac{\sum_i (x_i - \overline{x})(y_i - \overline{y})}{\sqrt{\sum_i (x_i - \overline{x})^2} \cdot \sqrt{\sum_i (y_i - \overline{y})^2}}$$

the values from -1 to $+1$ measure the strength of a straight-line relationship only

Reading this in words: for each customer, see how far their x is above or below the average x, and how far their y is above or below the average y, and multiply those two together. If x and y tend to be above average together (and below together), the products are positive and r comes out positive. The denominator is just bookkeeping that squeezes the answer into the range -1 to $+1$. The crucial limitation, hidden in the word "straight", is that Pearson only sees **straight-line** relationships.

pearson.py — python

```python
sub = df[["income", "monthly_charge"]].dropna()
r = stats.pearsonr(sub.income, sub.monthly_charge)
print(f"Pearson r = {r.statistic:.3f}  (p = {r.pvalue:.1e})")
```

Output

```
Pearson r = 0.144  (p = 9.6e-04)
```

Pearson reports a feeble 0.144, which would normally be read as "income and monthly charge are barely related". Hold that thought, because it is about to be exposed as misleading. (The p-value, 0.00096, is a separate idea: it estimates how likely we would see a correlation this size purely by chance. Small p-values mean "probably not just luck". We will not dwell on it.)

5.2 Spearman correlation: do they move together at all

The problem with Pearson is that a handful of extreme values can wreck it. **Spearman correlation**, written ρ ("rho"), fixes this with a clever trick: instead of using the raw values, it replaces every value by its **rank** (1st, 2nd, 3rd, and so on) and then measures the straight-line relationship of the ranks. Ranks do not care how extreme a value is, only its position in the order, so Spearman is immune to outliers and to curved-but-consistent relationships.

$$\rho = r_{\text{Pearson}}(\text{rank}(x), \text{rank}(y))$$

Spearman is just Pearson computed on the rankings instead of the raw values

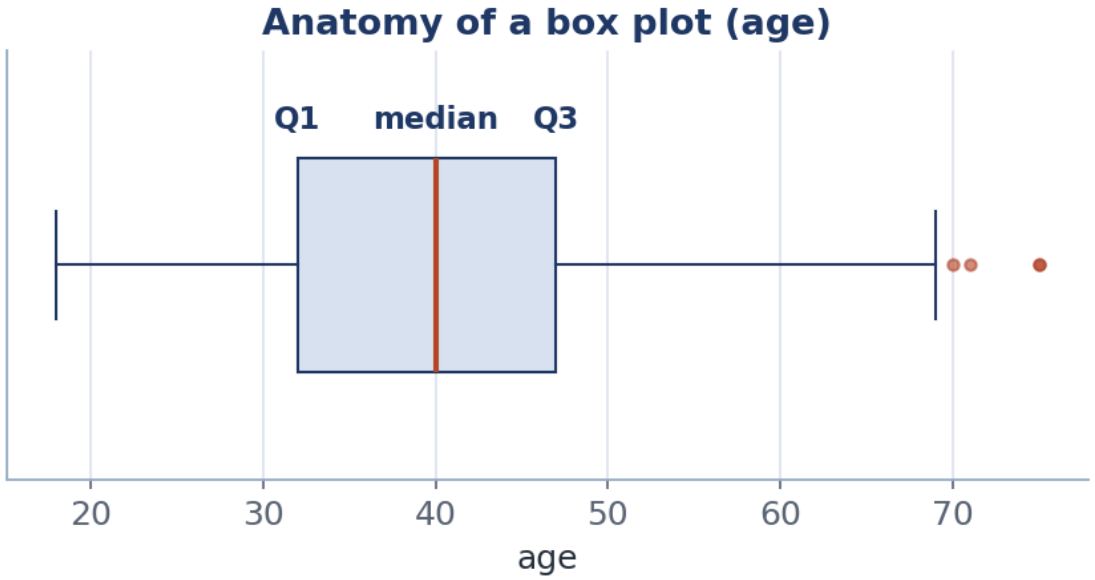
spearman.py python

```python
rho = stats.spearmanr(sub.income, sub.monthly_charge)
print(f"Spearman rho = {rho.statistic:.3f}  (p = {rho.pvalue:.1e})")
```

Output

```
Spearman rho = 0.821  (p = 2.1e-128)
```

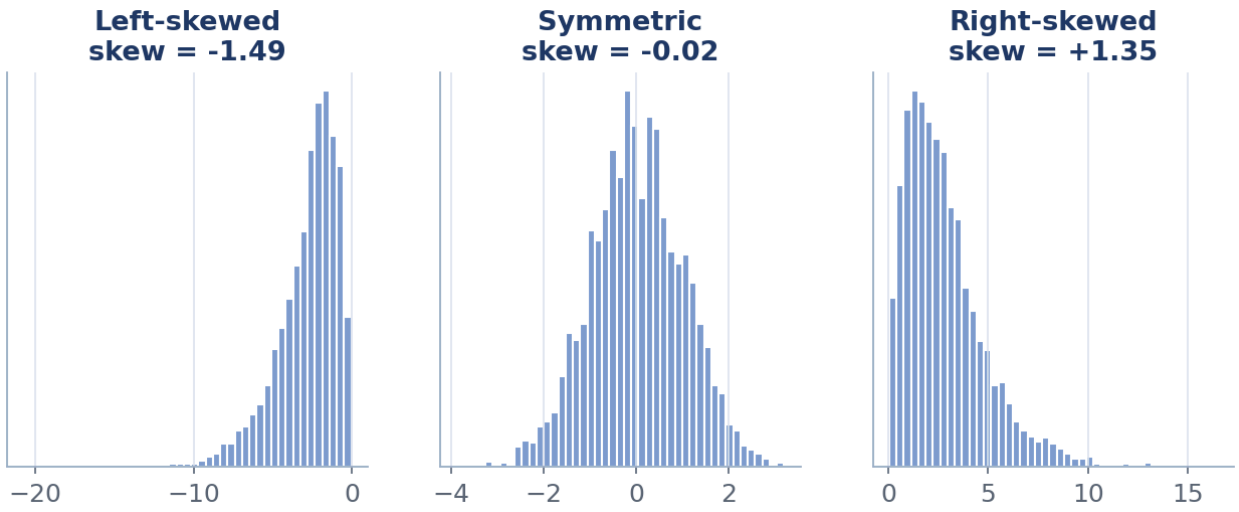
The same two columns, two very different stories
Pearson said 0.14, "barely related". Spearman says 0.82, "strongly related". These are the exact same two columns. What happened? A few customers with absurd incomes (our planted outliers) threw off Pearson's straight-line view, but Spearman, working on ranks, simply saw that higher income reliably goes with higher charge. The lesson: when in doubt, report both, and when they disagree, draw the scatter plot and trust your eyes.



A relationship that climbs steadily but along a curve, not a straight line. Pearson under-reports the connection because it is looking for a straight line; Spearman, working on ranks, reports it correctly.

5.3 Anscombe's quartet: the reason we always plot

In 1973 a statistician named Francis Anscombe built four little datasets that share the same mean, the same spread, the same correlation, and even the same best-fit line. On paper they look identical. Drawn out, they could not be more different. It remains the single most persuasive argument for never trusting a summary number without looking at the data.



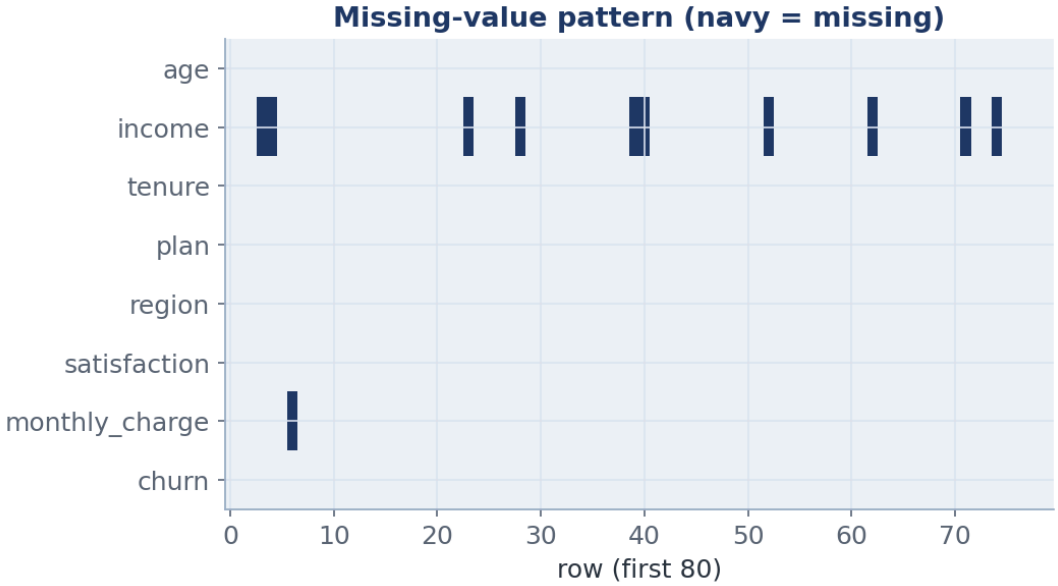
Four datasets with identical summary statistics and an identical fitted line. One is a tidy linear trend, one is a curve, one is a straight line with a single outlier, and one is almost all the same x-value. Same numbers, four different realities.

5.4 Looking at everything at once

When you have many columns, you do not check pairs one at a time; you compute the whole **correlation matrix** and colour it in. Each cell shows how strongly two columns move together.

corr.py — python

```python
num = df.select_dtypes("number")        # keep only the numeric columns
print(num.corr().round(2))              # Pearson by default; use .corr("spearman") for ranks
```



A correlation heatmap. The diagonal is always 1 (every column correlates perfectly with itself). Off the diagonal, red means "rise together", blue means "one rises as the other falls", and pale means "little relationship". Here income and monthly_charge stand out gently from the rest.

5.5 Correlation is not causation

This deserves its own short section because it is the most common mistake in all of data analysis. Two things moving together does **not** mean one causes the other.

> In summer, ice-cream sales rise and so do drownings. They are strongly correlated. But ice cream does not cause drowning. A third thing, hot weather, causes both. Whenever you see a correlation, ask: could one cause the other, could something else cause both, or could it just be chance in a small sample? Treat correlation as a clue worth investigating, never as proof.

6 Data processing

Processing (also called preprocessing) is the cleaning-and-chopping step: turning raw, imperfect data into something a model can actually use. This chapter covers the five most common chores: deciding how data arrives, putting columns on a comparable scale, dealing with blanks, handling extreme values, and, when there is simply too little data, manufacturing more of it through augmentation. One golden rule runs through all of it, and we will meet it properly in Chapter 7: whatever you learn from the data, learn it from the **training** portion only.

6.1 Batch versus stream: how the data arrives

There are two fundamentally different situations. In **batch** processing you have a fixed pile of data sitting in a file, and you work through all of it together. It is repeatable, easy to check, and it is what we use for coursework and for training most models. In **stream** processing the data keeps arriving, like a live feed of transactions, so you can only hold a little at a time, you must update your calculations as you go, and you have to watch for the pattern slowly changing over time (called **drift**). Deciding which situation you are in shapes every later choice, so decide it early.

6.2 Feature scaling: putting columns on the same ruler

Many models measure distances between data points or take small steps guided by the numbers' sizes. If one column is in the tens (age, 18 to 85) and another is in the tens of thousands (income), the big-numbered column drowns out the small one purely because of its units, not its importance. **Scaling** fixes this by rescaling every column to a comparable range.

$$z = \frac{x - \mu}{\sigma} \quad \text{(standardisation, the z-score)} \quad ; \quad x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad \text{(min-max)}$$

standardisation re-centres a column to average 0 and spread 1; min-max squashes it into the range 0 to 1

In words: **standardisation** (the "z-score") subtracts the mean so the column is centred on zero, then divides by the standard deviation so its spread becomes one. A z-score of +2 means "two standard deviations above average". **Min-max** scaling instead stretches or squashes the column so its smallest value becomes 0 and its largest becomes 1. Both make different columns comparable.

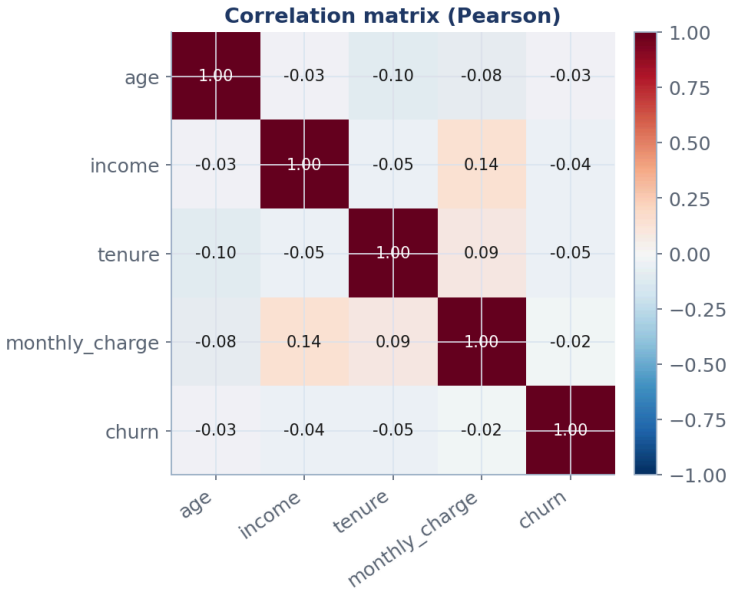
scale.py — python

```python
from sklearn.preprocessing import StandardScaler
X = df[["age", "monthly_charge"]].dropna()
print(StandardScaler().fit_transform(X)[:3].round(2))
```

Output

```
[[ 0.47  1.09]
 [-0.78 -0.87]
 [ 0.86  1.35]]      # each number is now "how many standard deviations from average"
```

After scaling, both columns speak the same language. A value of 0.47 means "a bit above average"; −0.78 means "a fair way below average". The raw units (years, euros) have been removed, so no column can dominate just because its numbers happen to be large.



On the left, monthly_charge is invisible because age's larger numbers dominate the axis. On the right, after standardisation, both columns are centred at 0 with a comparable spread, so a model can weigh them fairly.

6.3 Missing data: why blanks happen and what to do

Real data has holes. The right way to handle a blank depends on **why** it is blank, and there are three classic cases. The names are jargon, so here is the plain version of each.

- **MCAR** (missing completely at random): the blank has nothing to do with anything, like a sensor that randomly drops a reading. Dropping those rows is unbiased, just wasteful.
- **MAR** (missing at random): the blank is explained by something you **did** record, for example younger users more often skip the income question. Because you know their age, you can sensibly fill the gap using the other columns.
- **MNAR** (missing not at random): the blank is explained by the very value that is missing, for example high earners deliberately hiding their income. This is the hard case; no simple fill is fully honest, and you should at least flag it.

> **A blank can itself be a clue**
> Before you fill a gap, consider adding a simple yes-or-no column that records "this value was missing". The fact that someone refused to answer can itself predict behaviour, and erasing it throws that signal away.

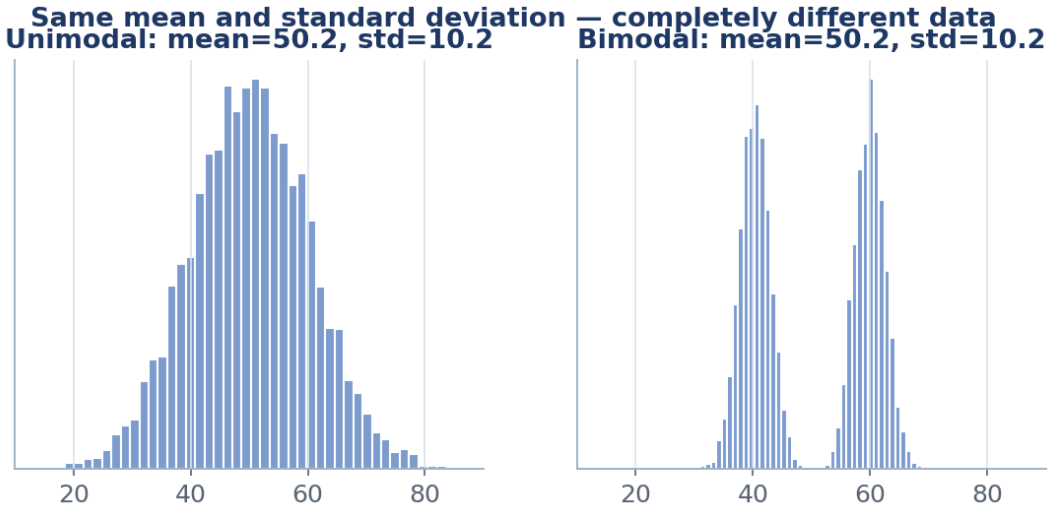
We first **look** at where the holes are, rather than blindly filling them.

missing.py — python

```python
print(df.isna().sum())              # count blanks in each column
```

Output

```
age               0
income           53
monthly_charge   30
plan / region     0
churn             0
```



A missing-value map of the first 80 customers, one row per column. Navy marks a blank. Seeing the pattern (are the gaps scattered or clustered together?) tells you more than a single count ever could.

Once we understand the gaps, we **impute** them, which is a fancy word for "fill them in with a sensible guess". The simplest sensible guess is the median (robust to outliers). Smarter methods like **KNN imputation** look at the most similar customers and borrow their values.

$$\hat{x}_{\text{miss}} = \text{median}(x_{\text{observed}})$$

the simplest fill: replace each blank with the median of the values we do have

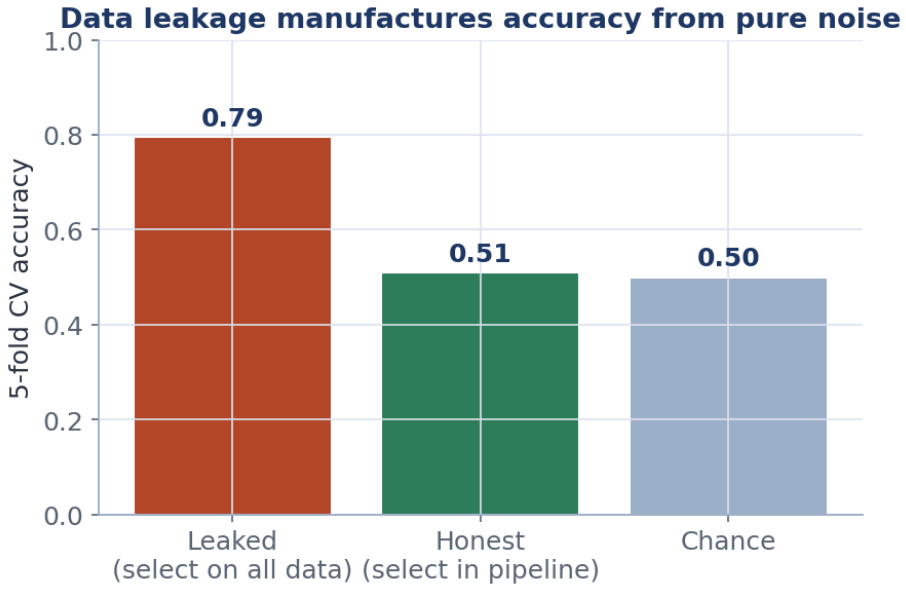
impute.py — python

```python
from sklearn.impute import SimpleImputer, KNNImputer
median = SimpleImputer(strategy="median").fit_transform(df[["income"]])
knn    = KNNImputer(n_neighbors=5).fit_transform(df[["age", "income", "tenure"]])
print("missing after median fill:", int(np.isnan(median).sum()))
```

Output

```
missing after median fill: 0
```

The count of blanks is now 0, so every gap has been filled. But filling is not free, as the next picture shows.



Median filling puts every single blank onto exactly one value, creating an artificial spike (the tall red bar) that did not exist in the real data and that shrinks the apparent spread. KNN imputation, which borrows from similar customers, spreads the fills more realistically. Always know what your fill is doing to the shape.

6.4 Outliers: spotting the absurd values

An **outlier** is a value that sits far away from the rest, like our planted incomes of over a million. They can be genuine (a real billionaire), a data-entry error (an extra zero), or a sign that you have mixed two different populations together. Two simple rules flag them automatically.

$$|z_i| = \frac{|x_i - \overline{x}|}{s} > 3 \qquad \text{or} \qquad x < Q_1 - 1.5 \cdot \text{IQR} \quad \text{or} \quad x > Q_3 + 1.5 \cdot \text{IQR}$$

the z-rule: more than 3 standard deviations from the mean. The IQR rule: outside the whiskers of the box plot.

In words: the **z-score rule** says "flag anything more than three standard deviations from the average", which works well if the column is roughly a bell curve. The **IQR rule** says "flag anything below Q1 minus 1.5 IQRs, or above Q3 plus 1.5 IQRs", which is exactly the box plot's whiskers and needs no assumption about the shape. The IQR rule is the safer default for messy data.

outliers.py — python

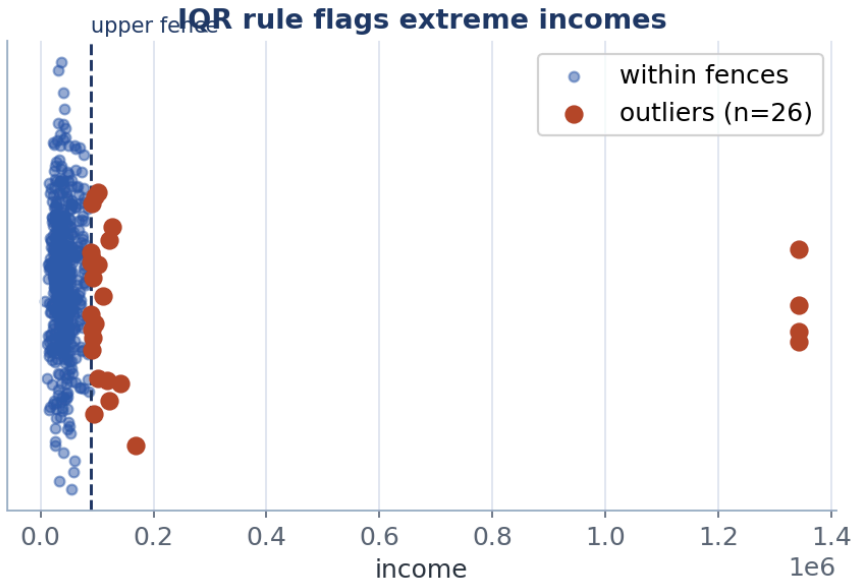
```python
q1, q3 = inc.quantile([.25, .75]); iqr = q3 - q1
hi = q3 + 1.5*iqr                       # the upper whisker
flagged = inc[inc > hi]
print(f"upper fence = {hi:,.0f}")
print(f"flagged     = {len(flagged)};  max = {flagged.max():,.0f}")
```

Output

```
upper fence = 88,154
flagged     = 26;  max = 1,342,344
```

The rule draws a line at about 88,000 and flags 26 incomes above it, the largest being our absurd 1.3 million. Flagging is not the same as deleting. The right response depends on the cause: fix obvious

typos, keep rare-but-real values (perhaps treating them separately), and never delete a value just because it is inconvenient. Understand it first.



The same incomes spread along a line. Blue points sit within the normal range; the red points beyond the dashed fence are the flagged outliers. The cluster on the far right is our planted absurd values, now clearly separated from the bulk of customers.

6.5 Data augmentation: making more when you have too little

So far every problem in this chapter has been about data that is messy. There is a different problem that no amount of cleaning can fix: not having **enough** data in the first place. A model learns a general rule by seeing many varied examples. Give it only a handful, and instead of learning the rule it simply memorises the few rows it was shown. It then scores brilliantly on those rows and falls apart on anything new. This failure is called **overfitting**, and scarce data is one of its most common causes.

The trouble bites hardest for the rare cases you often care about most. In our own customers table, only 91 of the 600 customers actually churned, so the "left" class is outnumbered roughly five to one. A model can score 85 percent accuracy just by predicting "stayed" for everyone, while being completely useless at the one thing we wanted: spotting the customers about to leave. Too few examples of the important class is a data-quantity problem in disguise.

Before reaching for anything clever, it is worth knowing the full menu of responses to "not enough data", roughly from best to cheapest: collect more and better data (almost always the best fix, but slow and often expensive); reuse a model already trained on a larger dataset, which is **transfer learning**, a Week 9 to 11 topic; keep the model simple and add **regularisation** so it cannot memorise; and finally **data augmentation**, the subject of this section, which manufactures extra training examples out of the ones you already have.

> Imagine teaching a child to recognise a cat from just three photographs, all taken from the front. Show them the same three photos flipped left-to-right, tilted a little, and lit differently, and they suddenly have far more to learn from, without you finding a single new cat. That is augmentation: you make new, believable examples from the ones you have, as long as each change leaves the **answer** untouched. A cat flipped left-to-right is still a cat.

The one rule: transformations must preserve the label

Augmentation works by applying a **label-preserving transformation**: a change that produces a new, realistic example whose correct answer is exactly the same as the original's. This one rule is the

whole game. If a change could alter the right answer, it is not augmentation, it is corruption. The classic cautionary tale is images of handwritten digits: flipping a photo of a "6" upside-down turns it into a "9", so that particular transformation quietly teaches the model something false. Always ask, "does the label still hold after this change?" before you use it.

Techniques depend on the kind of data

What counts as a realistic, label-preserving change depends entirely on what the data is. The table below lists the common families.

Kind of data	Label-preserving transformations you can apply
Images	Flip, rotate a little, crop, zoom, shift colour or brightness, add noise, erase a small patch
Text	Swap in synonyms, back-translate (translate out and back), randomly insert, delete, or swap words
Tabular (like ours)	SMOTE (interpolate new rows for a rare class), add small random noise (jitter), sample from a fitted model
Audio and time-series	Shift in time, change pitch or speed, add background noise, gently warp the timing
Generative (any type)	Learn the whole distribution, then sample fresh examples: VAEs, GANs, diffusion models

A formula or two, in words

Two tabular techniques have simple formulas worth seeing. The first is **jitter**: nudge each numeric value by a small random amount, scaled to the column's own spread, so the new row is a believable neighbour of a real one.

$$x' = x + \varepsilon, \quad \varepsilon \sim \mathcal{N}\big(0, (\alpha\sigma)^2\big)$$

add a little Gaussian noise whose size is a small fraction alpha of the column's own standard deviation sigma

Read this as: take a real value x, add a small random wobble ε drawn from a bell curve centred on zero, and keep the wobble small by tying its size to the column's spread σ through a little fraction α (say 0.05). The result x' is a new value close to a real one.

The second, and the most used for rare classes in tabular data, is **SMOTE** (Synthetic Minority Over-sampling Technique). Instead of copying rare rows, which teaches nothing new, SMOTE invents believable ones **between** existing rare rows.

$$x_{\text{new}} = x_i + \lambda\big(x_{z_i} - x_i\big), \quad \lambda \sim \text{Uniform}(0, 1)$$

pick a rare row, pick one of its near neighbours, and step a random fraction of the way from one to the other

Read this as: take a real minority example x_i, find one of its nearest neighbours x_{z_i} that is also in the minority, and place a brand-new synthetic point somewhere on the straight line between them, at a random fraction λ of the distance. The new point looks like it belongs to the rare group, because it sits right among genuine members of it.

Worked example: balancing our churn data

Our churn column is imbalanced, so it is a natural place to use SMOTE. The crucial detail, which we will justify in full in Chapter 7, is that we **split first** and only ever augment the **training** rows. The test set must stay untouched and real, or our scores become fiction.

augment.py python

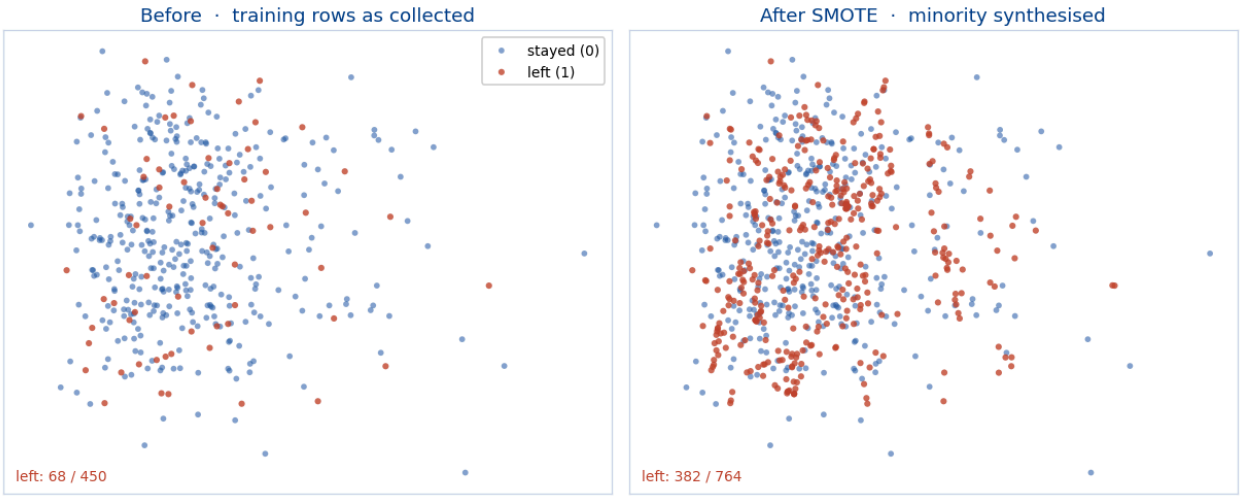
```python
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from imblearn.over_sampling import SMOTE

num = ["age", "income", "tenure", "monthly_charge"]
X = SimpleImputer(strategy="median").fit_transform(df[num])   # SMOTE needs numbers, no blanks
y = df["churn"]

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25,
                                          random_state=0, stratify=y)  # SPLIT FIRST
print("train before:", dict(Counter(y_tr)))
X_aug, y_aug = SMOTE(random_state=0).fit_resample(X_tr, y_tr)         # augment TRAIN only
print("train after :", dict(Counter(y_aug)))
```

```
$ python augment.py
```

Output

```
train before: {0: 382, 1: 68}      # the 'left' class is heavily outnumbered
train after : {0: 382, 1: 382}     # SMOTE invented 314 synthetic 'left' customers
```

The rare class has been lifted from 68 rows to 382, matching the majority, by inventing 314 synthetic customers that sit among the real churners. The training set has grown from 450 rows to 764, and, importantly, the test set was never touched. A model trained on the balanced set now has a fair chance of learning what a churning customer looks like.



Left: the scarce, outnumbered "left" customers (red) among the many who stayed (blue). Right: SMOTE has filled the gaps between real red points with synthetic red points, so the rare class is no longer a scattered handful. Only the training rows are augmented; the test rows stay exactly as collected.

Augment the training set only, and after the split
This is the same leakage trap as the rest of preprocessing, and it is easy to fall into. If you augment **before** splitting, synthetic rows built from a test customer can leak into training, and your scores lie. If you augment the **test** set, you are grading the model on made-up data. The safe recipe is always: split first, then augment the training portion only, ideally inside a pipeline (Chapter 7) so it can never happen by accident. Note we use `imblearn`'s pipeline for this, because resampling must run on training folds only.

Learned augmentation: generative models

The techniques above tweak or interpolate existing rows. A more powerful family instead **learns the whole shape of your data** and then invents fresh examples from that learned shape. These are the same generative models you will meet later in the module, now put to work making data.

- A **variational autoencoder** (VAE) squeezes each example down to a small set of numbers (a "latent" code), learns how those codes are distributed, then samples new codes and expands them back into realistic examples.
- A **generative adversarial network** (GAN) pits two networks against each other: one invents fake examples, the other tries to spot the fakes, and they improve together until the fakes are convincing. For tables specifically, CTGAN is a popular variant.
- **Diffusion models** start from pure noise and remove it a little at a time until a realistic example emerges. They currently produce the most convincing images.
- For plain tables you can reach for ready-made tools such as CTGAN or TVAE (in the SDV library); for text, a large language model can paraphrase an example while keeping its meaning and label.

These methods are strong but demanding. They need a reasonable amount of data to train in the first place, they can quietly **amplify whatever bias is already in the data**, and they can invent combinations that never occur in reality. The same golden rule still applies: train the generator on the training split only, and always look at a sample of what it produces before trusting it.

When not to augment, and what it cannot do

Augmentation is powerful but it is not magic, and it has honest limits. It can only recombine and perturb the variety already present; it cannot invent a kind of customer, or a group of people, that your data never contained. So if a whole population is missing from your dataset, augmentation will not conjure them, and leaning on it can hide that gap rather than fix it. Too much augmentation, or transformations that are not quite realistic, can inject noise and actually hurt. And synthetic minority rows from SMOTE can blur the boundary between classes if the rare group is very sparse or riddled with outliers. Treat augmentation as a way to stretch genuinely representative data further, never as a substitute for collecting the examples you are truly missing. When in doubt, prefer more real data.

7 Leakage-safe pipelines

This chapter is about the single most expensive mistake in machine learning, one that fools even experienced practitioners. It is called **data leakage**. Understanding it is worth more than any fancy algorithm, because a model built with leakage looks brilliant in testing and then fails the moment it meets the real world.

7.1 What leakage is, in plain terms

> Leakage is like letting a student see the exam answer key while they revise, then being amazed when they ace the practice exam. Of course they did well; they had already seen the answers. The practice score tells you nothing about how they will do on a real, unseen exam. Data leakage is the same thing: information from your test data sneaks into the training process, the test score looks fantastic, and then the model collapses on genuinely new data.

The commonest way leakage happens is innocent. You decide to scale your columns, or fill blanks, or pick the best features, and you do it on the **whole** dataset before splitting off a test set. But "the whole dataset" includes the test set, so your preparation has quietly peeked at the answers. The cure is mechanical and absolute: **split first, then learn every preparation step from the training portion only, and bundle the whole thing into one object called a pipeline.**

7.2 A demonstration you will not forget

To show how dangerous leakage is, we run a deliberately rigged experiment. We create 200 fake customers described by 5000 columns of **pure random noise**, and we assign each a random label. There is, by construction, absolutely nothing to learn here. An honest model must score about 50 percent, the same as flipping a coin.

First, the wrong way. We pick the 20 "best-looking" columns using **all** the data, and only then do we test.

leak_wrong.py — python

```python
import numpy as np
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

g = np.random.default_rng(0)
X, y = g.normal(size=(200, 5000)), g.integers(0, 2, size=200)   # pure noise

Xsel = SelectKBest(f_classif, k=20).fit_transform(X, y)         # peeks at ALL the data
print(cross_val_score(LogisticRegression(max_iter=1000), Xsel, y, cv=5).mean())
```

```
$ python leak_wrong.py
```

Output

```
0.795       # 80 percent accuracy on pure noise. That is impossible, and a lie.
```

Eighty percent accuracy on data that contains no pattern at all. The "feature selection" step, by looking at the whole dataset including the test folds, found columns that happened to match the answers by luck, and handed that luck to the model. This is leakage in action.

Now the honest way. The only change is that the column-picking step lives **inside** a pipeline, so it is re-done from scratch on each training fold and never sees the test fold.

leak_right.py python

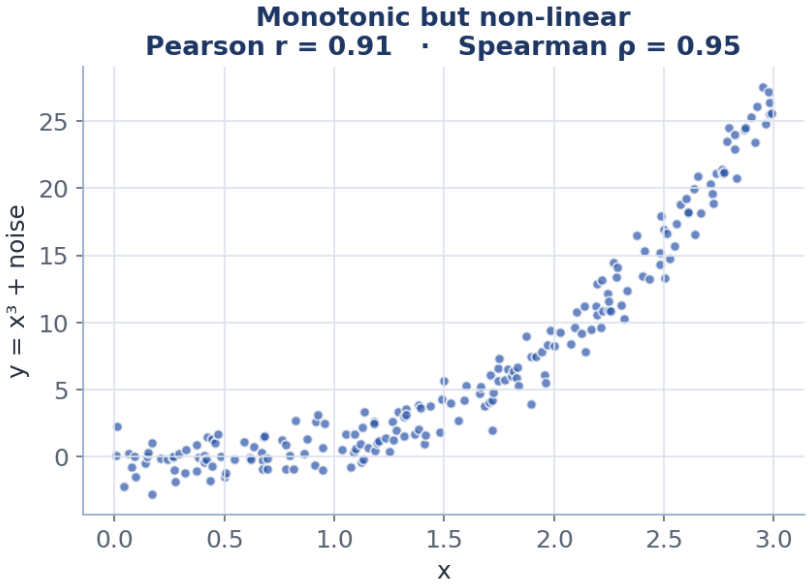
```python
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ("select", SelectKBest(f_classif, k=20)),     # now this is refit per training fold
    ("clf",    LogisticRegression(max_iter=1000)),
])
print(cross_val_score(pipe, X, y, cv=5).mean())
```

```
$ python leak_right.py
```

Output

```
0.510       # about 50 percent: chance, exactly as it should be. The truth.
```

The honest score is 51 percent, essentially a coin flip, which is correct because there was never anything to learn. The only difference between a believable result and a complete fiction was one detail: whether the preparation peeked at the test data.



The same code, two stories. Moving the feature-selection step inside the pipeline turns a fictional 0.79 into the honest 0.51. The grey bar is pure chance for reference.

7.3 A reusable, leakage-safe template

The good news is that doing this correctly is not hard; you just have to wire it up once. The pattern below bundles all preparation into a `ColumnTransformer` (which applies different steps to different columns) and then wraps that together with the model in a single `Pipeline`. Because you split the data first and only ever call `fit` on the training part, every step automatically learns from training data alone. No leakage is possible.

pipeline.py python

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

num = ["age", "income", "tenure", "monthly_charge"]    # numeric columns
cat = ["plan", "region", "satisfaction"]               # word columns

prep = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),   # fill blanks
                      ("scale",  StandardScaler())]),                  # then scale
                      num),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat),             # encode words
])
model = Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=1000))])

X = df[num + cat]; y = df["churn"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=0)  # split FIRST
model.fit(X_tr, y_tr)                  # every step learns from the training rows only
print("test accuracy:", round(model.score(X_te, y_te), 3))
```

This one object is what you carry forward into Week 2. To try a different model, you swap only the last line of the pipeline and leave everything else untouched. It is already leakage-safe, which means the scores it gives you can be trusted.

8 Data governance and documentation

A model is only as trustworthy as the data behind it and the records you keep about that data. **Governance** is the unglamorous but essential practice of writing down what your data is, where it came from, and what it can and cannot be used for. It is also where the ethics topics of Weeks 3 and 4 truly begin, because most unfairness in models is inherited from the data, not invented by the algorithm.

8.1 The data dictionary: explaining every column

A **data dictionary** is a plain table that describes every column: its name, what type it is, its units, its allowed values, what it means, where it came from, and any quirks a future user should know. It takes ten minutes to write and saves hours of confusion later.

Column	Type	Notes for a future user
age	continuous (years)	18 to 85; capped at collection time
income	continuous (euros)	lopsided; about 9 percent missing; contains extreme values
tenure	whole number (months)	0 to 71
plan	category	Basic, Standard, or Premium
region	category	North, South, East, or West
satisfaction	ordered category	Low, then Medium, then High
monthly_charge	continuous (euros)	about 5 percent missing
churn	yes/no (0 or 1)	the thing we predict; 1 means the customer left

8.2 The datasheet: explaining the whole dataset

Where the dictionary describes the columns, a **datasheet** describes the dataset as a whole. The idea comes from a well-known 2021 paper by Timnit Gebru and colleagues, and it works like a nutrition label for data. It answers questions such as:

- **Why does this dataset exist**, and who made it?
- **What is in it**: who or what does each row represent, who is included, and importantly, who is left out?
- **How was it collected**: when, by whom, and with what consent from the people in it?
- **What was already done to it**: any cleaning, labelling, or filtering before you received it?
- **What may it be used for**: the licence, and any uses that would be inappropriate or harmful.

8.3 From documentation to ethics

Here is why this matters beyond tidiness. The biases you will study in Weeks 3 and 4 are usually baked into the data long before any model exists: in who got sampled, in how the labels were decided, in which questions were even asked. A careful data dictionary and datasheet are the cheapest, earliest defence against building something unfair, because they force you to notice who is missing and what assumptions are hiding in the data. That is why both are a graded part of your Week 1 lab and your final project.

A Formula cheat-sheet

Every formula from the booklet, gathered in one place for quick reference.

Quantity	Definition
Mean	$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$
Variance (sample)	$s^2 = \frac{1}{n-1}\sum_{i=1}^{n} (x_i - \bar{x})^2$
Standard deviation	$s = \sqrt{s^2}$
Interquartile range	$\mathrm{IQR} = Q_3 - Q_1$
Skewness	$g_1 = \left(\frac{1}{n}\sum (x_i - \bar{x})^3\right) / s^3$
Excess kurtosis	$g_2 = \left(\frac{1}{n}\sum (x_i - \bar{x})^4\right) / s^4 - 3$
Standardisation (z-score)	$z = (x - \mu)/\sigma$
Min-max scaling	$x' = (x - x_{\min})/(x_{\max} - x_{\min})$
Pearson correlation	$r = \left(\sum (x_i - \bar{x})(y_i - \bar{y})\right) / \left(s_x s_y (n-1)\right)$
Spearman correlation	$\rho = r(\mathrm{rank}(x), \mathrm{rank}(y))$
Outlier fences (IQR rule)	$Q_1 - 1.5 \cdot \mathrm{IQR}$ and $Q_3 + 1.5 \cdot \mathrm{IQR}$
Normal distribution	$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)}$
Jitter (augmentation)	$x' = x + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, (\alpha\sigma)^2)$
SMOTE (augmentation)	$x_{\text{new}} = x_i + \lambda\left(x_{z_i} - x_i\right), \quad \lambda \sim \mathrm{Uniform}(0,1)$

B Running everything yourself

One script, `make_assets.py`, reproduces every number and figure in this booklet. From the project folder, run the three lines below.

terminal python

```
python -m venv .venv && source .venv/bin/activate     # make and enter a clean workspace
pip install numpy pandas scikit-learn scipy matplotlib
python make_assets.py                                  # writes figs-week01/ and prints every
result
```

It prints each labelled result (the means, the correlations, the encodings, the imputation counts, the outlier fences, and the leakage comparison) and saves every chart into a `figs-week01` folder. The best way to build real understanding is to change one thing at a time, for example the random seed, the amount of skew, or how many blanks to poke, then run it again and watch how the numbers and pictures respond.

C Glossary

A quick reference for the jargon used in this booklet, each in one plain sentence.

Term	Plain meaning
EDA	Exploratory data analysis: carefully looking at data before modelling
Distribution	How common each value is in a column
Mean / median	The average / the middle value
Standard deviation	A measure of how spread out the values are
IQR	The range covering the middle half of the data
Skewness	How lopsided a column is
Outlier	A value far away from the rest
MCAR / MAR / MNAR	The three reasons a value can be missing
Imputation	Filling in a missing value with a sensible guess
Encoding	Turning words into numbers a model can use
One-hot	Encoding a category as one yes/no column per option
Standardisation	Rescaling a column to average 0 and spread 1
Correlation	A number for how strongly two columns move together
Leakage	Test-set information accidentally reaching the model during training
Overfitting	Memorising the training rows instead of learning the general pattern
Augmentation	Manufacturing extra training examples with label-preserving changes
SMOTE	Building synthetic rows for a rare class by interpolating between real ones
VAE / GAN	Generative models that learn a dataset's shape and sample brand-new examples
Pipeline	One object that bundles all preparation and the model, fit together safely
Datasheet	A nutrition-label-style record of a dataset's origin and limits

Going further. W. McKinney, *Python for Data Analysis* (the pandas library, from its creator). · J. VanderPlas, *Python Data Science Handbook* (exploration and visualisation). · T. Gebru and colleagues, "Datasheets for Datasets", *Communications of the ACM* 64(12), 2021. · N. V. Chawla and colleagues, "SMOTE: Synthetic Minority Over-sampling Technique", *Journal of Artificial Intelligence Research* 16, 2002. · F. J. Anscombe, "Graphs in Statistical Analysis", *The American Statistician* 27(1), 1973. · The scikit-learn User Guide, especially the chapters on Preprocessing and Pipelines.